

MYSQL STORED PROCEDURES

Examples and Practice Exercises for MySQL Workbench

1. Setup: Employees Table

The screenshot displays the MySQL Workbench interface. The left sidebar contains the 'MANAGEMENT' section with options like Server Status, Client Connections, and Users and Privileges. The 'INSTANCE' section includes Startup / Shutdown, Server Logs, and Options File. The 'PERFORMANCE' section includes Dashboard, Performance Reports, and Performance Schema Setup. The 'Administration' tab is selected, showing 'Schemas' and 'Information'. The 'Information' section shows 'No object selected'. The 'Query' tab is active, displaying a SQL script to create a database, use it, create a table, and insert data. The 'Output' tab shows the execution results of the script.

MySQL Workbench

Local instance MySQL80 x Mysql@127.0.0.1:3306 x

File Edit View Query Database Server Tools Scripting Help

Navigator: Query 1 x

MANAGEMENT

- Server Status
- Client Connections
- Users and Privileges
- Status and System Variables
- Data Export
- Data Import/Restore

INSTANCE

- Startup / Shutdown
- Server Logs
- Options File

PERFORMANCE

- Dashboard
- Performance Reports
- Performance Schema Setup

Administration Schemas

Information: No object selected

Query 1 x

```
1 • CREATE DATABASE IF NOT EXISTS employeeedb1;
2 • use employeeedb1;
3 • CREATE TABLE IF NOT EXISTS employees (
4     emp_id INT PRIMARY KEY,
5     emp_name VARCHAR(50),
6     dept_id INT,
7     salary DECIMAL(10,2)
8 );
9 • INSERT INTO employees VALUES
10 (101,'Rahul',1,50000),
11 (102,'Priya',2,65000),
12 (103,'Anil',1,55000),
13 (104,'Sneha',3,70000),
14 (105,'Kiran',2,48000);
15
```

SQLAdditions: Automatic context help is disabled. Use the toolbar to manual the current caret position or to toggle automatic h

Context Help Snippets

Output: Action Output

#	Time	Action	Message
2	10:35:46	show databases	12 row(s) returned
3	10:36:44	use db2	0 row(s) affected
4	11:44:41	CREATE DATABASE IF NOT EXISTS employeeedb1	1 row(s) affected
5	11:45:26	use employeeedb1	0 row(s) affected
6	11:45:56	CREATE TABLE IF NOT EXISTS employees (emp_id INT PRIMARY KEY, e...	0 row(s) affected
7	11:46:20	INSERT INTO employees VALUES (101,'Rahul',1,50000), (102,'Priya',2,65000), (10...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0

Object Info Session

2. Verify Data

Query 1 x SQLA

Limit to 1000 rows

```

9 • INSERT INTO employees VALUES
10   (101, 'Rahul', 1, 50000),
11   (102, 'Priya', 2, 65000),
12   (103, 'Anil', 1, 55000),
13   (104, 'Sneha', 3, 70000),
14   (105, 'Kiran', 2, 48000);
15 • select * from employees;

```

Result Grid

	emp_id	emp_name	dept_id	salary
▶	101	Rahul	1	50000.00
	102	Priya	2	65000.00
	103	Anil	1	55000.00
	104	Sneha	3	70000.00
	105	Kiran	2	48000.00
*	NULL	NULL	NULL	NULL

Result Grid

Form Editor

Procedure 1 – Display All Employees

Question: Create a procedure that displays all employees

Query 1 x

Limit to 1000 rows

```

2 • DROP PROCEDURE IF EXISTS GetEmployees;
3   DELIMITER //
4 • CREATE PROCEDURE GetEmployees()
5   BEGIN
6     SELECT * FROM employees;
7   END //
8   DELIMITER ;
9 • CALL GetEmployees();
10

```

Result Grid

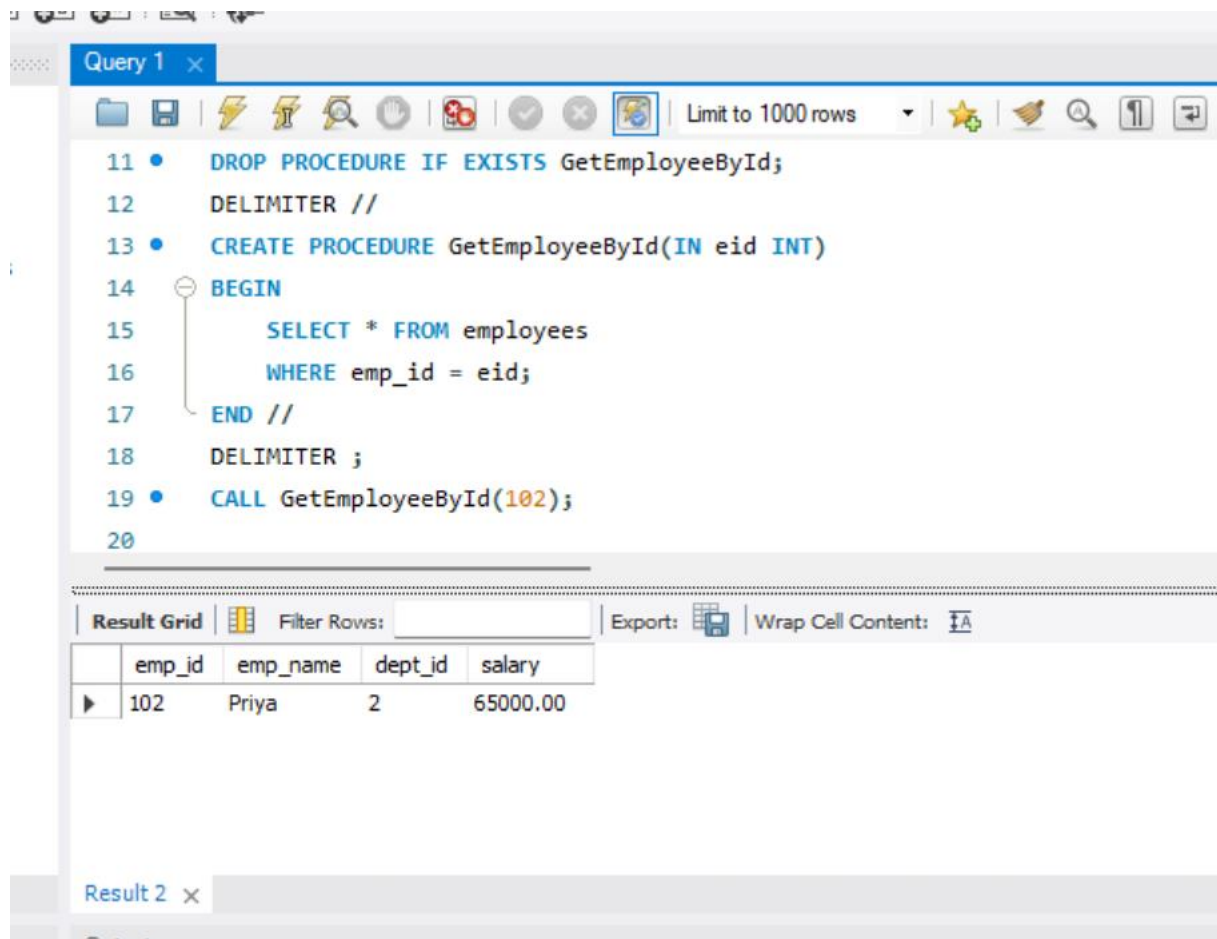
	emp_id	emp_name	dept_id	salary
▶	101	Rahul	1	50000.00
	102	Priya	2	65000.00
	103	Anil	1	55000.00
	104	Sneha	3	70000.00
	105	Kiran	2	48000.00

Export: Wrap Cell Content: [IA](#)

Result 1 x

Procedure 2 – IN Parameter

Question: Create a procedure that accepts an employee ID and displays that employee's details.



The screenshot shows a SQL IDE window titled "Query 1". The SQL code is as follows:

```
11 • DROP PROCEDURE IF EXISTS GetEmployeeById;
12   DELIMITER //
13 • CREATE PROCEDURE GetEmployeeById(IN eid INT)
14   BEGIN
15       SELECT * FROM employees
16       WHERE emp_id = eid;
17   END //
18   DELIMITER ;
19 • CALL GetEmployeeById(102);
20
```

Below the code editor, the "Result Grid" is displayed, showing the output of the procedure call. The grid has columns for emp_id, emp_name, dept_id, and salary. The first row shows the details for employee ID 102.

emp_id	emp_name	dept_id	salary
102	Priya	2	65000.00

At the bottom of the window, there is a tab labeled "Result 2".

Procedure 3 – Update Salary

Question: Create a procedure that accepts employee ID and amount, then increases the employee's salary.

Query 1

```

21 • DROP PROCEDURE IF EXISTS IncreaseSalary;
22 DELIMITER //
23 • CREATE PROCEDURE IncreaseSalary(
24     IN eid INT,
25     IN amount DECIMAL(10,2)
26 )
27 BEGIN
28     UPDATE employees
29     SET salary = salary + amount
30     WHERE emp_id = eid;

```

Result Grid

emp_id	emp_name	dept_id	salary
101	Rahul	1	55000.00
NULL	NULL	NULL	NULL

Query 1

```

26 )
27 BEGIN
28     UPDATE employees
29     SET salary = salary + amount
30     WHERE emp_id = eid;
31 END //
32 DELIMITER ;
33 • CALL IncreaseSalary(101,5000);
34 • SELECT * FROM employees WHERE emp_id=101;
35

```

Result Grid

emp_id	emp_name	dept_id	salary
101	Rahul	1	55000.00
NULL	NULL	NULL	NULL

Procedure 4 – Aggregate Function

Question: Create a procedure to calculate the average salary of all employees.

bles

```

35
36 • DROP PROCEDURE IF EXISTS AvgSalary;
37 DELIMITER //
38 • CREATE PROCEDURE AvgSalary()
39 BEGIN
40     SELECT AVG(salary) AS AverageSalary
41     FROM employees;
42 END //
43 DELIMITER ;
44 • CALL AvgSalary();

```

tup

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	AverageSalary
▶	58600.000000

Procedure 5 – JOIN

Question: Create a procedure to display employee name, department name and salary. Run the department setup first.

```

45
46 • CREATE TABLE IF NOT EXISTS departments (
47     dept_id INT PRIMARY KEY,
48     dept_name VARCHAR(50)
49 );
50
51 • INSERT IGNORE INTO departments VALUES
52     (1, 'CSE'), (2, 'ECE'), (3, 'EEE');
53
54 • DROP PROCEDURE IF EXISTS EmployeeDepartment;
55
56 DELIMITER //
57 • CREATE PROCEDURE EmployeeDepartment()
58 BEGIN
59     SELECT e.emp_name, d.dept_name, e.salary
60     FROM employees e
61     JOIN departments d ON e.dept_id=d.dept_id;
62 END //

```

56 DELIMITER //

57 • CREATE PROCEDURE EmployeeDepartment()

58 BEGIN

59 SELECT e.emp_name, d.dept_name, e.salary

60 FROM employees e

61 JOIN departments d ON e.dept_id=d.dept_id;

62 END //

63 DELIMITER ;

64

65 • CALL EmployeeDepartment();

Result Grid | Filter Rows: | Export: | Wrap Cell Content: IA

	emp_name	dept_name	salary
▶	Rahul	CSE	55000.00
	Priya	ECE	65000.00
	Anil	CSE	55000.00
	Sneha	EEE	70000.00
	Kiran	ECE	48000.00

Result 6 x

Procedure 6 – GROUP BY and AVG

Question: Create a procedure to display department-wise employee count and average salary.

67 • DROP PROCEDURE IF EXISTS DepartmentAverageSalary;

68 DELIMITER //

69 • CREATE PROCEDURE DepartmentAverageSalary()

70 BEGIN

71 SELECT d.dept_name,

72 COUNT(e.emp_id) AS EmployeeCount,

73 AVG(e.salary) AS AverageSalary

74 FROM employees e

75 JOIN departments d ON e.dept_id=d.dept_id

76 GROUP BY d.dept_id, d.dept_name;

Result Grid | Filter Rows: | Export: | Wrap Cell Content: IA

	dept_name	EmployeeCount	AverageSalary
▶	CSE	2	55000.000000
	ECE	2	56500.000000
	EEE	1	70000.000000

ables

```
73         AVG(e.salary) AS AverageSalary
74     FROM employees e
75     JOIN departments d ON e.dept_id=d.dept_id
76     GROUP BY d.dept_id, d.dept_name;
77 END //
78 DELIMITER ;
79
80 • CALL DepartmentAverageSalary();
81
82
```

etup

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

	dept_name	EmployeeCount	AverageSalary
▶	CSE	2	55000.000000
	ECE	2	56500.000000
	EEE	1	70000.000000

Result 7 x

Procedure 7 – IF...ELSE

Question: Display High Salary if salary is at least 60000; otherwise display Low Salary.

The screenshot shows the SQL Developer interface with a query window titled "Query 1". The query contains the following SQL code:

```
82 • DROP PROCEDURE IF EXISTS SalaryStatus;
83   DELIMITER //
84 • CREATE PROCEDURE SalaryStatus(IN eid INT)
85   BEGIN
86     DECLARE sal DECIMAL(10,2);
87     SELECT salary INTO sal
88     FROM employees
89     WHERE emp_id=eid;
90
91     IF sal >= 60000 THEN
```

Below the query editor, the "Result Grid" tab is active, showing a single column header "Status" and one row with the value "High Salary".

The screenshot shows the SQL Developer interface with a query window titled "Query 1". The query contains the following SQL code:

```
90
91   IF sal >= 60000 THEN
92     SELECT 'High Salary' AS Status;
93   ELSE
94     SELECT 'Low Salary' AS Status;
95   END IF;
96 END //
97 DELIMITER ;
98 • CALL SalaryStatus(102);
99
```

Below the query editor, the "Result Grid" tab is active, showing a single column header "Status" and one row with the value "High Salary".

Procedure 8 – OUT Parameter

Question: Return the total number of employees using an OUT parameter.

The screenshot shows the SQL Server Enterprise Manager interface. On the left, a tree view displays 'Databases', 'Tables', 'Views', 'Stored Procedures', 'Functions', 'System Variables', 'Restore', and 'Shutdown'. The main window is titled 'Query 1' and contains the following SQL code:

```

2 • DROP PROCEDURE IF EXISTS EmployeeCount;
3   DELIMITER //
4 • CREATE PROCEDURE EmployeeCount(OUT total INT)
5   BEGIN
6       SELECT COUNT(*) INTO total
7       FROM employees;
8   END //
9   DELIMITER ;
10 • CALL EmployeeCount(@total);
11 • SELECT @total AS TotalEmployees;

```

Below the query window, the 'Result Grid' tab is active, showing a single row with the column 'TotalEmployees' and the value '5'.

Procedure 9 – INOUT Parameter

Question: Receive a bonus through an INOUT parameter and add 5000 to it.

The screenshot shows the SQL Server Enterprise Manager interface. On the left, a tree view displays 'Databases', 'Tables', 'Views', 'Stored Procedures', 'Functions', 'System Variables', 'Restore', and 'Shutdown'. The main window is titled 'Query 1' and contains the following SQL code:

```

13 • DROP PROCEDURE IF EXISTS AddBonus;
14   DELIMITER //
15 • CREATE PROCEDURE AddBonus(INOUT bonus DECIMAL(10,2))
16   BEGIN
17       SET bonus = bonus + 5000;
18   END //
19   DELIMITER ;
20 • SET @bonus=10000;
21 • CALL AddBonus(@bonus);
22 • SELECT @bonus AS UpdatedBonus;

```

Below the query window, the 'Result Grid' tab is active, showing a single row with the column 'UpdatedBonus' and the value '15000.00'.

Procedure 10 – Transaction

Question: Transfer an amount from one employee salary record to another using START TRANSACTION and COMMIT.

es

```

24 • DROP PROCEDURE IF EXISTS TransferSalaryAmount;
25 DELIMITER //
26 • CREATE PROCEDURE TransferSalaryAmount(
27     IN fromEmp INT,
28     IN toEmp INT,
29     IN amount DECIMAL(10,2)
30 )
31 BEGIN
32     START TRANSACTION;
33     UPDATE employees

```

Result Grid

emp_id	emp_name	dept_id	salary
101	Rahul	1	53000.00
102	Priya	2	67000.00
NULL	NULL	NULL	NULL

employees 11 x

Query 1 x

```

34 SET salary=salary-amount
35 WHERE emp_id=fromEmp;
36 UPDATE employees
37 SET salary=salary+amount
38 WHERE emp_id=toEmp;
39 COMMIT;
40 END //
41 DELIMITER ;
42 • CALL TransferSalaryAmount(101,102,2000);
43 • SELECT * FROM employees WHERE emp_id IN (101,102);

```

Result Grid

emp_id	emp_name	dept_id	salary
101	Rahul	1	53000.00
102	Priya	2	67000.00
NULL	NULL	NULL	NULL

employees 11 x

Output

Action Output

#	Time	Action	Message
---	------	--------	---------

4. Useful Procedure Commands

bles

```
43 • SELECT * FROM employees WHERE emp_id IN (101,102);
44
45 • SHOW PROCEDURE STATUS
46   WHERE Db='employeeedb1';
47
48 • SHOW CREATE PROCEDURE GetEmployees;
49
50 • DROP PROCEDURE IF EXISTS GetEmployees;
51
52
```

tup

Db	Name	Type	Definer	Modified	Created	Secur
employeeedb1	IncreaseSalary	PROCEDURE	root@localhost	2026-08-08 12:03:53	2026-08-08 12:03:53	DEFIN
employeeedb1	SalaryStatus	PROCEDURE	root@localhost	2026-08-08 12:29:08	2026-08-08 12:29:08	DEFIN
employeeedb1	TransferSalaryAmount	PROCEDURE	root@localhost	2026-08-08 12:29:08	2026-08-08 13:36:11	DEFIN

Result 12

6. Student Practice Tasks

1. Create a procedure to display employees whose salary is greater than a given amount.

```
2 DELIMITER //
3 • CREATE procedure employee_above_salary(IN amount decimal(10,2))
4 BEGIN
5   Select * From employees
6   where salary > amount;
7 END //
8 DELIMITER ;
9 • CALL employee_above_salary(50000);
```

emp_id	emp_name	dept_id	salary
101	Rahul	1	53000.00
102	Priya	2	67000.00
103	Anil	1	55000.00
104	Sneha	3	70000.00

2. Create a procedure to update an employee's department using IN parameters.

```

DELIMITER //
• CREATE PROCEDURE update_department(
    IN p_emp_id INT,
    IN p_dept_id INT
)
• BEGIN
    UPDATE employees
    SET dept_id = p_dept_id
    WHERE emp_id = p_emp_id;
END //
DELIMITER ;
• CALL update_department(101,3);
• select * from employees;

```

```

34 • CALL update_department(101,3);
35 • select * from employees;

```

up

Result Grid				
Filter Rows:				
Edit: Export/Import:				
	emp_id	emp_name	dept_id	salary
▶	101	Rahul	3	53000.00
	102	Priya	2	67000.00
	103	Anil	1	55000.00
	104	Sneha	3	70000.00
	105	Kiran	2	48000.00
*	NULL	NULL	NULL	NULL

3. Create a procedure to return the maximum salary using an OUT parameter.

```

40
41
42
43 DELIMITER //
44 • CREATE PROCEDURE max_salary(OUT max_sal DECIMAL(10,2))
45 • BEGIN
46     SELECT MAX(salary)
47     INTO max_sal
48     FROM employees;
49 END //
50 DELIMITER ;
51 • CALL max_salary(@max);
52 • SELECT @max;

```

up

Output

Action Output

```

43 DELIMITER ;
44 • CALL max_salary(@max);
45 • SELECT @max;

```

Result Grid		Filter Rows:	Export:
	@max		
▶	70000.00		

4. Create a procedure to return the employee count of a specified department.

```

46
47 DELIMITER //
48 • CREATE PROCEDURE employee_count(
49     IN p_dept_id INT,
50     OUT emp_count INT
51 )
52 BEGIN
53     SELECT COUNT(*)
54     INTO emp_count
55     FROM employees
56     WHERE dept_id = p_dept_id;
57 END //
58 DELIMITER ;
59 • CALL employee_count(1, @count);
60 • SELECT @count;

```

```

54     INTO emp_count
55     FROM employees
56     WHERE dept_id = p_dept_id;
57 END //
58 DELIMITER ;
59 • CALL employee_count(1, @count);
60 • SELECT @count;

```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	@count			
▶	1			

5. Create a procedure using JOIN to display employee and department details.

The screenshot shows a database IDE with a SQL editor and a result grid. The SQL editor contains the following code:

```
62 DELIMITER //
63 • CREATE PROCEDURE employee_department()
64 BEGIN
65     SELECT e.emp_id, e.emp_name, e.salary, d.dept_name
66     FROM employees e
67     JOIN departments d
68     ON e.dept_id = d.dept_id;
69 END //
70 DELIMITER ;
71 • CALL employee_department();
```

The result grid displays the output of the procedure, showing a list of employees and their departments:

emp_id	emp_name	salary	dept_name
101	Rahul	53000.00	EEE
102	Priya	67000.00	ECE
103	Anil	55000.00	CSE
104	Sneha	70000.00	EEE
105	Kiran	48000.00	ECE

Result 8 x

6. Create a procedure to display departments whose average salary is greater than 55000.

The screenshot shows a database IDE with a SQL editor and a result grid. The SQL editor contains the following code:

```
72 DELIMITER //
73 • CREATE PROCEDURE high_avg_salary()
74 BEGIN
75     SELECT dept_id, AVG(salary) AS avg_salary
76     FROM employees
77     GROUP BY dept_id
78     HAVING AVG(salary) > 55000;
79 END //
80 DELIMITER ;
81 • CALL high_avg_salary();
```

The result grid displays the output of the procedure, showing departments with an average salary greater than 55000:

dept_id	avg_salary
3	61500.000000
2	57500.000000

Result 9 x

7. Create a procedure using IF...ELSE to classify salary as High, Medium or Low.


```
83 DELIMITER //
84 • CREATE PROCEDURE classify_salary(IN p_emp_id INT)
85 BEGIN
86     DECLARE sal DECIMAL(10,2);
87
88     SELECT salary INTO sal
89     FROM employees
90     WHERE emp_id = p_emp_id;
91
92     IF sal >= 60000 THEN
93         SELECT 'High' AS salary_category;
94     ELSEIF sal >= 40000 THEN
95         SELECT 'Medium' AS salary_category;
96     ELSE
97         SELECT 'Low' AS salary_category;
98     END IF;
99 END //
100 DELIMITER ;
```

Output :

```
92 IF sal >= 60000 THEN
93     SELECT 'High' AS salary_category;
94 ELSEIF sal >= 40000 THEN
95     SELECT 'Medium' AS salary_category;
96 ELSE
97     SELECT 'Low' AS salary_category;
98 END IF;
99 END //
```

```
100 DELIMITER ;
101 • CALL classify_salary(101);
```

Result Grid

salary_category
Medium

8. Create a procedure that accepts a department ID and returns its average salary.

```
101 • CALL classify_salary(101);
102
103 DELIMITER //
104 • CREATE PROCEDURE avg_salary(
105     IN p_dept_id INT,
106     OUT avg_sal DECIMAL(10,2)
107 )
108 • BEGIN
109     SELECT AVG(salary)
110     INTO avg_sal
111     FROM employees
112     WHERE dept_id = p_dept_id;
113 END //
114 DELIMITER ;
115 • CALL avg_salary(1, @avg);
116 • SELECT @avg;
```

Output

bles

```
108 • BEGIN
109     SELECT AVG(salary)
110     INTO avg_sal
111     FROM employees
112     WHERE dept_id = p_dept_id;
113 END //
114 DELIMITER ;
115 • CALL avg_salary(1, @avg);
116 • SELECT @avg;
```

Output

Result Grid	Filter Rows:	Ex
@avg		
55000.00		

9. Create a procedure using INOUT to apply a bonus to an input salary.

117
118 DELIMITER //

119 • CREATE PROCEDURE apply_bonus(INOUT sal DECIMAL(10,2))

120 BEGIN

121 SET sal = sal + (sal * 10 / 100);

122 END //

123 DELIMITER ;

124 • SET @salary = 50000;

125 • CALL apply_bonus(@salary);

126 • SELECT @salary;

Result Grid

	@salary
▶	55000.00

Setup

10. Create a procedure that performs two related updates inside a transaction.

128 DELIMITER //

129 • CREATE PROCEDURE update_employee()

130 BEGIN

131 START TRANSACTION;

132

133 UPDATE employees

134 SET salary = salary + 5000

135 WHERE emp_id = 101;

136

137 UPDATE employees

138 SET dept_id = 2

139 WHERE emp_id = 101;

140

141 COMMIT;

142 END //

143 DELIMITER ;

144 • CALL update_employee();

es

```
137      UPDATE employees
138      SET dept_id = 2
139      WHERE emp_id = 101;
140
141      COMMIT;
142  END //
143  DELIMITER ;
144 • CALL update_employee();
145 • select * from employees;
146
```

p

Result Grid   Filter Rows: Edit:   

	emp_id	emp_name	dept_id	salary
▶	101	Rahul	2	58000.00
	102	Priya	2	67000.00
	103	Anil	1	55000.00
	104	Sneha	3	70000.00
	105	Kiran	2	48000.00

employees 14 ×